

# Proyek Sistem Operasi

## Minggu 4: Multithreading & Sinkronisasi POSIX

Novalio Daratha  
Program Studi Informatika  
Universitas Bengkulu

2026

# Outline Minggu 4

# Proses vs Thread (Lightweight Process)

## Proses

- Ruang alamat terisolasi (*isolated address space*).
- *Overhead* pembuatan dan *context switch* relatif besar.
- Komunikasi data harus melalui IPC.

## Thread

- Berbagi *code*, *data*, dan *heap* dalam proses yang sama.
- Masing-masing thread memiliki *stack* dan *registers* sendiri.
- *Overhead* sangat ringan dan cepat.

# Membuat Thread dengan pthread\_create()

```
#include <pthread.h>
#include <stdio.h>

void* worker(void* arg) {
    int id = *(int*)arg;
    printf("Thread %d sedang berjalan...\n", id);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);

    pthread_join(t1, NULL); // Tunggu t1 selesai
    pthread_join(t2, NULL); // Tunggu t2 selesai
    return 0;
}
```

# Kondisi Balapan (*Race Condition*)

## Definisi Race Condition

Situasi di mana beberapa thread mengakses dan memanipulasi data bersama secara konkuren, dan hasil akhirnya bergantung pada urutan eksekusi thread.

## Critical Section

Segmen kode di mana thread mengakses variabel bersama (*shared resource*). Harus memenuhi 3 syarat:

- 1 **Mutual Exclusion:** Hanya 1 thread yang boleh berada di Critical Section pada satu waktu.
- 2 **Progress:** Thread di luar Critical Section tidak boleh menghalangi thread lain.
- 3 **Bounded Waiting:** Tidak boleh ada thread yang kelaparan (*starvation*) selamanya.

# Sinkronisasi: Mutex Lock

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);    // Masuk Critical Section
        counter++;
        pthread_mutex_unlock(&lock);  // Keluar Critical Section
    }
    return NULL;
}
```

## Hasil

Variabel counter dijamin tepat bernilai 200,000 jika dijalankan oleh 2 thread secara paralel.

# Deadlock: 4 Kondisi Coffman

## Deadlock Terjadi Jika & Hanya Jika Keempat Kondisi Terpenuhi:

- 1 **Mutual Exclusion:** Resource bersifat non-shareable.
- 2 **Hold and Wait:** Thread menahan setidaknya 1 resource sambil menunggu resource lain.
- 3 **No Preemption:** Resource tidak bisa diambil paksa dari thread yang menahannya.
- 4 **Circular Wait:** Ada rantai melingkar saling menunggu ( $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_1$ ).

# Ada Pertanyaan?

Praktikum: Buka worksheet4.pdf untuk simulasi race condition dan implementasi Mutex.